

Academic Question Answering System Using RAG

Retrieval-Augmented Generation with PDF Page Citations

Abstract

The Academic Question Answering System uses Retrieval-Augmented Generation (RAG) to answer questions from a selected collection of academic PDFs. The system extracts text while retaining page information, divides the content into overlapping chunks, creates semantic embeddings, and stores them in a vector database. When a question is submitted, relevant passages are retrieved and provided to a language model to generate an answer grounded in the supplied documents. Source and page citations help users verify the evidence behind each response.

The project focuses on making academic information easier to find while improving answer traceability. Its reliability is assessed through tests for retrieval, answer quality, and citation correctness.

1. Technology Stack

The project uses a Python-based RAG pipeline. The following components reflect the current implementation direction and project workflow.

Python

Core language used to connect document processing, retrieval, generation, and evaluation.

PyMuPDF

Extracts selectable text from PDFs and supports page-level source tracking.

LangChain Recursive Character Text Splitter

Splits extracted content into manageable overlapping chunks while preserving useful context.

Sentence Transformers — BAAI/bge-small-en-v1.5

Converts chunks and user questions into embeddings for semantic search.

ChromaDB

Stores embeddings, chunk text, and metadata for persistent vector retrieval.

Google GenAI / Gemini API

Generates answers using the passages retrieved from the indexed PDFs.

python-dotenv

Loads configuration and API credentials from environment variables rather than hard-coding secrets.

Supporting project elements

- PDF metadata includes the source filename and original page number.
- Chunking configuration currently discussed: approximately 250 tokens per chunk with 60 tokens of overlap.
- Evaluation is run through the project's evaluation workflow; exact metrics should be reported only when measured.

The documented stack is limited to the components established in the project workflow. Other libraries or interface frameworks are not listed as implemented unless they are present in the project.

2. Flow and Architecture

The system has two main workflows: preparing the document knowledge base and answering a user's question.

A. Document indexing flow

Academic PDF files → PDF text extraction → text cleaning → recursive chunking → page/source metadata → embeddings → persistent ChromaDB collection

During ingestion, the original filename and page number must travel with each chunk. This metadata is essential for generating citations that point back to the source PDF.

B. Question-answering flow

User question → question embedding → vector similarity search → top relevant chunks → prompt with retrieved context → Gemini response → answer with source/page citations

C. Main architecture components

- **Ingestion layer:** reads PDF content and captures page-level metadata.
- **Preprocessing layer:** cleans text and creates overlapping chunks.
- **Embedding layer:** represents chunks and questions as vectors.
- **Vector database:** stores and retrieves semantically similar chunks.
- **Generation layer:** uses retrieved evidence to formulate the response.
- **Citation layer:** displays source and page information from stored metadata.
- **Evaluation layer:** checks whether retrieval, answers, and citations meet expected behavior.

Citation principle

Citations should be generated from stored metadata, not invented by the language model. If the indexed documents do not contain enough evidence, the system should communicate that limitation instead of presenting an unsupported answer.

3. Directory Structure

The directory layout below reflects the document-processing and evaluation workflow established for the project. Keep each stage separate so that extracted text, cleaned text, chunks, and evaluation data can be inspected independently.

```
Academic-RAG-System/  
■■■ extract_text.py  
■■■ chunk_text.py  
■■■ answer.py  
■■■ evaluate_rag.py  
■■■ data/  
    ■■■ raw_pdfs/  
    ■■■ extracted_text/  
    ■■■ cleaned_text/  
    ■■■ processed_chunks/  
    ■■■ evaluation/  
    ■■■ chroma_db/
```

Important files

- **extract_text.py**: extracts PDF text and preserves source/page information.
- **chunk_text.py**: cleans and splits extracted content into overlapping chunks.
- **answer.py**: connects question processing, retrieval, response generation, and citation output.
- **evaluate_rag.py**: runs evaluation checks on the RAG workflow.
- **data/raw_pdfs/**: stores the original academic PDFs.
- **data/extracted_text/** and **data/cleaned_text/**: hold intermediate text outputs for inspection.
- **data/processed_chunks/**: holds prepared chunk data and metadata.
- **data/evaluation/**: holds evaluation questions or evaluation artefacts.
- **data/chroma_db/**: stores the persistent ChromaDB index.

Directory names should match the actual repository. If the implementation uses different names or combines stages into fewer files, update this tree to reflect the codebase exactly.

4. Testing and Performance

The project has reported a successful evaluation status and passed tests after the latest changes. The checks below describe the behaviors that should be covered; measured timings or accuracy figures should be added only when available.

Testing coverage

- **PDF extraction:** extracted page count matches the source and text is readable.
- **Metadata:** each retrieved chunk retains a valid source filename and original page number.
- **Relevant question:** the system retrieves supporting passages and produces a grounded answer.
- **Paraphrased question:** semantic retrieval finds relevant evidence despite different wording.
- **Unsupported question:** the system avoids fabricating information when evidence is absent.
- **Citation validation:** cited pages exist and support the answer.
- **Error handling:** empty input, missing documents, or API errors are handled clearly.

Performance evaluation

- **Retrieval quality:** use Precision@K, Recall@K, or ranking measures on a labelled question set.
- **Answer quality:** assess factual correctness, completeness, and faithfulness to retrieved passages.
- **Citation quality:** measure whether citations are valid and support the claims they accompany.
- **Latency:** record indexing time and average question-response time over repeated trials.

5. Future Scope

- Add OCR support for scanned PDFs and improve extraction of tables and equations.
- Explore hybrid keyword-plus-vector retrieval and reranking to improve passage selection.
- Add highlighted source passages and clickable page-level citations.
- Expand the dataset to more subjects and create a larger, independently labelled evaluation set.
- Improve handling of multi-document questions, conversation history, and unsupported queries.
- Add a deployment-ready interface, usage monitoring, and robust error handling where needed.

6. Conclusion

The project combines PDF extraction, chunking, embeddings, vector search, and language-model generation to answer academic questions from supplied documents. Preserving source and page metadata makes responses easier to verify, while testing and evaluation help reveal weaknesses in retrieval, answer faithfulness, and citation accuracy. Future improvements should be guided by measured results on a representative question set.